



**JOMO KENYATTA UNIVERSITY
OF
AGRICULTURE & TECHNOLOGY**

SCHOOL OF OPEN, DISTANCE AND eLEARNING

P.O. Box 62000, 00200

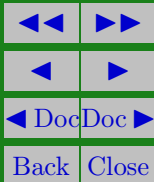
Nairobi, Kenya

E-mail: elearning@jkuat.ac.ke

ICS 2104 Object-Oriented Programming I

JKUAT SODEL

©2017





ICS 2104 Object-Oriented Programming I

This presentation is intended to be covered within one week. The notes, examples and exercises should be supplemented with a good textbook. Most of the exercises have solutions/answers appearing elsewhere and accessible by clicking the green **Exercise** tag. To move back to the same page click the same tag appearing at the end of the solution/answer.

Errors and omissions in these notes are entirely the responsibility of the author who should only be contacted through the **Department of Curricula & Delivery (SODeL)** and suggested corrections may be e-mailed to elarning@jkuat.ac.ke.



LESSON 10

Files and Streams

Learning Outcomes

Upon completing this topic, you should be able to:

- Define a file and a stream
- Differentiate an input file stream and an output file stream
- Use streams to process data in files

10.1. Introduction

Many programs use large data sets that is stored in secondary memory

The data is stored in files.



ICS 2104 Object-Oriented Programming I

There are two basic types of files:

- Text files: The data in the file are characters
- Binary files: The data in the file are in binary form

Files can also be grouped into:

- Sequential files: The data is accessed in the order in which it is stored
- Random files: The data is accessed randomly

10.2. Stream

A stream is a sequence of characters. It is the one that links a program to a file. In object-oriented programs, stream are objects. In C++ streams are objects of type `fstream`. `fstream` is the name of a class that is stored in `fstream.h` file.



ICS 2104 Object-Oriented Programming I

An object of type `fstream` is declared as shown below:

```
fstream inputFile;
```

```
fstream outputFile;
```

where:

`fstream` is a keyword

`inputFile` and `outputFile` are names of the streams that are to be linked to files.

Connecting a file with a stream

Connecting a file to a stream is known as opening a file,

- using a constructor function
- using open member function

10.3. opening a file using constructor function.

The syntax of opening a file using constructor is:

JKUAT: Setting trends in higher Education, Research and Innovation



ICS 2104 Object-Oriented Programming I

```
fstream FstreamName("file_name", opening_mode);
```

where:

`fstream` is a reserved word

`file_name` is the name of the file being opened

`opening_mode` specifies whether a stream is being used for input, output or both

It can be one of the following specifiers:

- `ios::in` open for reading
- `ios::out` open for writing
- `ios::app` append at the end of the file
- `ios::ate` go to the end of the file
- `ios::binary` open as a binary file

For example:

JKUAT: Setting trends in higher Education, Research and Innovation



ICS 2104 Object-Oriented Programming I

```
fstream outfile("myfile1.cpp", ios::out); //output file,  
fstream infile("myfile2.cpp", ios::in); //input file.
```

Where

myfile1.cpp and myfile2.cpp are the names of files that are being opened.

ios::in and ios::out are file open modes for reading and writing respectively.

myfile1.cpp is opened for reading and myfile2.cpp is opened for writing

10.4. Using open() function

The syntax to open a file using open() function is

```
FstreamName.open("file_name", open_mode);
```

For example.



ICS 2104 Object-Oriented Programming I

```
fstream myfile; //line 1
```

```
myfile.open("Data1.cpp", ios::out); //line 2
```

where line 1 is a call to the default constructor in `fstream` class. `fstream` is a reserved word. In line 2 `open()` is a member function of `fstream` class. It is being called using `myfile` reference variable. `Data1.cpp` is the argument that is being passed to the `open()` function. It is the name of the file that is opened. `ios::out` is the file open mode. Note that the second argument when opening a file using either a constructor or an `open()` is file open mode.

A file can be opened in more than one file mode. This is done by combining modes using bitwise OR operator (`|`). For example `file1.open("myFile.cpp", ios::in|ios::binary);`



10.5. Checking for successful opening

It is always advisable to check whether a file has been opened successful before carrying out any operation on the file. There are many things that can make opening of a file to fail. For example, a file could be in a different directory or it could not be existing at all. Any attempt to carry out any operation on a file that has not been opened properly will fail.

`fail()` function is the one that is used to check whether a file has been opened successfully.

`fail()` function returns `true(1)` if the opening has failed and `false(0)` if the opening has succeeding

```
fstream inputfile("myfile.cpp", ios::in);
```

```
if( inputfile.fail())
```

```
//display message if the opening has failed
```



```
}
```

In such a case the program is terminated through the use of `exit()` function. This function is passed 1 to indicate that the program has not ended successfully.

```
if( inputfile.fail())  
    //display message if the opening has failed  
    exit(1);  
}
```

10.6. Closing a file

A file that has been opened should be closed after it has been processed. This is done through the use of `close()` function. This function disconnect a stream from a file that it has been connected with.



ICS 2104 Object-Oriented Programming I

Example:

```
fstream inputfile("myfile.cpp", ios::in);  
inputfile.close();
```

10.7. Input from and output to a file

Once a file has been connected to an input stream, the input operator `>>` can be used to input data from the file just like the way this operator is used to input data from a keyboard.

Example:

```
fstream inputfile("myfile.cpp", ios::in);  
inputfile>>variable1>>variable2>>...variablen;
```

Similarly, once a file has been connected to an output stream, the output operator `<<` can be used to output data to the file just like the way this operator is used to output data to a key-



ICS 2104 Object-Oriented Programming I

board.

Example:

```
fstream outputfile("myfile.cpp", ios::out);  
outputfile<<variable1<<variable2<<...variablen;
```

10.8. Detecting end of a file

It is always advisable to keep on checking whether one has reached the end of a file particularly when the file is an input. This is done using eof() function. This function returns true(nonzero) value when it has come to the end of a file and zero otherwise.



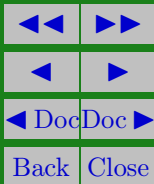
10.9. File pointers and function for manipulating file pointers

Each file has a get pointer and a put pointer. A get pointer enables a file to be read at certain point where a put pointer enables a file to be written at a certain point. When a file is opened for reading, the get pointer is automatically positioned at the beginning of the file. Similarly when a file is opened for writing the put pointer is positioned at the beginning of the file. These pointers are moved to any desired positions through the use of `seekg()`, `seekp()`, `tellg()` and `tellp()` functions. `seekg()` and `seekp()` moves the positions of get and put pointers respectively while `tellg()` and `tellp()` returns the current positions of get and put pointers respectively. `Seekp()` and `seekg()` are passed two arguments. The first argument is always the number of bytes



ICS 2104 Object-Oriented Programming I

that the pointer is to be moved, while the second argument is the position where the movement should begin. There are three constants that are used to denote the position where the movement should begin. These constants are: `ios::beg` //this means the movement should start at the beginning of the file `ios::cur` //this means the movement should start at the current position of the pointer `ios::end` //this means the movement should start at the end of the file Note, the movement can be either backward or forward starting at the specified position. This is done by preceding the number of bytes to be moved with either a plus(+)for forward or a minus (-) for backward. For example `inputfile.seekg(0, ios::beg);` //go to the beginning of the file `inputfile.seekg(m, ios::cur);` //move m bytes starting at the current position





Revision Questions

EXERCISE 1.  Define a stream

Example . With aid of examples, explain the difference between opening a file using a constructor and using `open()` function

Solution: `open()` function can be used to opening several files while a constructor can be used to open only one file at a time



EXERCISE 2.  Write a program that prompts a user to enter that name of a file. The program should display the content of the file on the screen

EXERCISE 3.  Write a program that counts and print the



ICS 2104 Object-Oriented Programming I

number of words in a file. The program should prompt a user to input the name of a file

EXERCISE 4.  Explain the various file opening mode



References and Additional Reading Materials

- David Flanagan, 2005, Java in a Nutshell (5th Edition), O'Reilly Press,
- Matt Weisfeld, 2009, The Object-Oriented Thought Process, 3rd Edition, , Addison-Wesley,
- Malik, D. S. 2002, C++ Programming: From Problem Analysis to Program Design, Course Technology, Canada
- Pappas, H. Chris and Murray, William, H. 1996, C/C++ Programmer's Guide, BPB Publications, New Delhi
- Flanagan, D. (2005). Java in a nutshell : a desktop quick reference. O'Reilly (5th ed.).
- Flanagan, D. (2004). Java examples in a nutshell : a tutorial companion to Java in a nutshell. O'Reilly (3rd ed.)



Solutions to Exercises

Exercise 1. A stream is a sequence of characters. It is a link between a file and a program. In object-oriented programs, stream are objects that connect programs with files. They are the ones that enables programs to read from or write to a file

Exercise 1